

```
In [1]: # import python libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt # visualizing data
%matplotlib inline
import seaborn as sns
```

```
In [3]: # import csv file
df = pd.read_csv(r'C:\Users\Dell\Downloads\Python_Diwali_Sales_Analysis-main (2)\Python_Diwali_Sales_Analysis-main\Diwali Sales Data.csv', encoding= 'unicode_escape')
```

```
In [4]: df.shape
```

```
Out[4]: (11251, 15)
```

```
In [6]: df.head(10)
```

```
Out[6]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe tthead th { text-align: right; } </style>
<thead>
<th></th>
<th>User_ID</th>
<th>Cust_name</th>
<th>Product_ID</th>
<th>Gender</th>
<th>Age Group</th>
<th>Age</th>
<th>Marital_Status</th>
<th>State</th>
<th>Zone</th>
<th>Occupation</th>
<th>Product_Category</th>
<th>Orders</th>
<th>Amount</th>
<th>Status</th>
<th>unnamed1</th>
</thead>
<tbody>
<th>0</th>
<th>1</th>
<th>2</th>
<th>3</th>
<th>4</th>
<th>5</th>
<th>6</th>
<th>7</th>
<th>8</th>
<th>9</th>
</tbody>
<tbody>
```

1002903	Sanskriti	P00125942	F	26-35	28	0	Maharashtra	Western	Healthcare	Auto	1	23952.00	NaN	NaN
1000732	Kartik	P00110942	F	26-35	35	1	Andhra Pradesh	Southern	Govt	Auto	3	23934.00	NaN	NaN
1001990	Bindu	P00118542	F	26-35	35	1	Uttar Pradesh	Central	Automobile	Auto	3	23924.00	NaN	NaN
1001425	Sudevi	P00237842	M	0-17	16	0	Karnataka	Southern	Construction	Auto	2	23912.00	NaN	NaN
1000588	Joni	P00057942	M	26-35	28	1	Gujarat	Western	Food Processing	Auto	2	23877.00	NaN	NaN
1000588	Joni	P00057942	M	26-35	28	1	Himachal Pradesh	Northern	Food Processing	Auto	1	23877.00	NaN	NaN
1001132	Balk	P00018042	F	18-25	25	1	Uttar Pradesh	Central	Lawyer	Auto	4	23841.00	NaN	NaN
1002092	Shivangi	P00273442	F	55+	61	0	Maharashtra	Western	IT Sector	Auto	1	NaN	NaN	NaN
1003224	Kushal	P00205642	M	26-35	35	0	Uttar Pradesh	Central	Govt	Auto	2	23809.00	NaN	NaN
1003650	Ginny	P00031142	F	26-35	26	1	Andhra Pradesh	Southern	Media	Auto	4	23799.99	NaN	NaN

```
In [13]: df.info()
```

```

<class 'pandas.DataFrame'>
Index: 11239 entries, 0 to 11250
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   User_ID          11239 non-null  int64
1   Cust_name        11239 non-null  str
2   Product_ID       11239 non-null  str
3   Gender           11239 non-null  str
4   Age Group        11239 non-null  str
5   Age              11239 non-null  int64
6   Marital_Status   11239 non-null  int64
7   State            11239 non-null  str
8   Zone             11239 non-null  str
9   Occupation       11239 non-null  str
10  Product_Category  11239 non-null  str
11  Orders           11239 non-null  int64
12  Amount           11239 non-null  float64
dtypes: float64(1), int64(4), str(8)
memory usage: 1.2 MB

```

```

In [8]: #drop unrelated/blank columns
df.drop(['Status', 'unnamed1'], axis=1, inplace=True)

```

```

In [14]: df.info()

```

```

<class 'pandas.DataFrame'>
Index: 11239 entries, 0 to 11250
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   User_ID          11239 non-null  int64
1   Cust_name        11239 non-null  str
2   Product_ID       11239 non-null  str
3   Gender           11239 non-null  str
4   Age Group        11239 non-null  str
5   Age              11239 non-null  int64
6   Marital_Status   11239 non-null  int64
7   State            11239 non-null  str
8   Zone             11239 non-null  str
9   Occupation       11239 non-null  str
10  Product_Category  11239 non-null  str
11  Orders           11239 non-null  int64
12  Amount           11239 non-null  float64
dtypes: float64(1), int64(4), str(8)
memory usage: 1.2 MB

```

```

In [10]: #check for null values
pd.isnull(df).sum()

```

```
Out[10]: User_ID      0
Cust_name      0
Product_ID     0
Gender         0
Age Group      0
Age            0
Marital_Status 0
State          0
Zone           0
Occupation     0
Product_Category 0
Orders         0
Amount        12
dtype: int64
```

```
In [11]: # drop null values
df.dropna(inplace=True)
```

```
In [12]: pd.isnull(df).sum()
```

```
Out[12]: User_ID      0
Cust_name      0
Product_ID     0
Gender         0
Age Group      0
Age            0
Marital_Status 0
State          0
Zone           0
Occupation     0
Product_Category 0
Orders         0
Amount         0
dtype: int64
```

```
In [18]: # change data type
df['Amount'] = df['Amount'].astype('int')
```

```
In [16]: df['Amount'].dtypes
```

```
Out[16]: dtype('float64')
```

```
In [17]: df.columns
```

```
Out[17]: Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age',
               'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category',
               'Orders', 'Amount'],
              dtype='str')
```

```
In [15]: #rename column
df.rename(columns= {'Marital_Status': 'Marriage Status'})
```

```
Out[15]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>User_ID</th>
<th>Cust_name</th> <th>Product_ID</th> <th>Gender</th> <th>Age Group</th> <th>Age</th> <th>Marriage Status</th> <th>State</th> <th>Zone</th> <th>Occupation</th>
<th>Product_Category</th> <th>Orders</th> <th>Amount</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>11246</th> <th>11247</th>
<th>11248</th> <th>11249</th> <th>11250</th> </tbody> <tbody></tbody>
1002903      Sanskriti P00125942 F 26-35 28 0      Maharashtra      Western      Healthcare      Auto 1 23952.0
1000732      Kartik P00110942 F 26-35 35 1      Andhra Pradesh      Southern      Govt      Auto 3 23934.0
1001990      Bindu P00118542 F 26-35 35 1      Uttar Pradesh      Central      Automobile      Auto 3 23924.0
1001425      Sudevi P00237842 M 0-17 16 0      Karnataka      Southern      Construction      Auto 2 23912.0
1000588      Joni P00057942 M 26-35 28 1      Gujarat      Western      Food Processing      Auto 2 23877.0
...      ...      ...      ...      ...      ...      ...      ...      ...      ...      ...      ...
1000695      Manning P00296942 M 18-25 19 1      Maharashtra      Western      Chemical      Office 4 370.0
1004089      Reichenbach P00171342 M 26-35 33 0      Haryana      Northern      Healthcare      Veterinary 3 367.0
1001209      Oshin P00201342 F 36-45 40 0      Madhya Pradesh      Central      Textile      Office 4 213.0
1004023      Noonan P00059442 M 36-45 37 0      Karnataka      Southern      Agriculture      Office 3 206.0
1002744      Brumley P00281742 F 18-25 19 0      Maharashtra      Western      Healthcare      Office 3 188.0

<p>11239 rows × 13 columns</p>
```

```
In [13]: # describe() method returns description of the data in the DataFrame (i.e. count, mean, std, etc)
df.describe()
```

```
Out[13]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>User_ID</th>
<th>Age</th> <th>Marital_Status</th> <th>Orders</th> <th>Amount</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th>
<th>max</th> </tbody> <tbody></tbody>
1.123900e+04 11239.000000 11239.000000 11239.000000 11239.000000
1.003004e+06 35.410357 0.420055 2.489634 9453.610553
1.716039e+03 12.753866 0.493589 1.114967 5222.355168
1.000001e+06 12.000000 0.000000 1.000000 188.000000
1.001492e+06 27.000000 0.000000 2.000000 5443.000000
1.003064e+06 33.000000 0.000000 2.000000 8109.000000
1.004426e+06 43.000000 1.000000 3.000000 12675.000000
1.006040e+06 92.000000 1.000000 4.000000 23952.000000
```

```
In [14]: # use describe() for specific columns
df[['Age', 'Orders', 'Amount']].describe()
```

```
Out[14]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Age</th>
<th>Orders</th> <th>Amount</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody>
</tbody>
```

11239.000000	11239.000000	11239.000000
35.410357	2.489634	9453.610553
12.753866	1.114967	5222.355168
12.000000	1.000000	188.000000
27.000000	2.000000	5443.000000
33.000000	2.000000	8109.000000
43.000000	3.000000	12675.000000
92.000000	4.000000	23952.000000

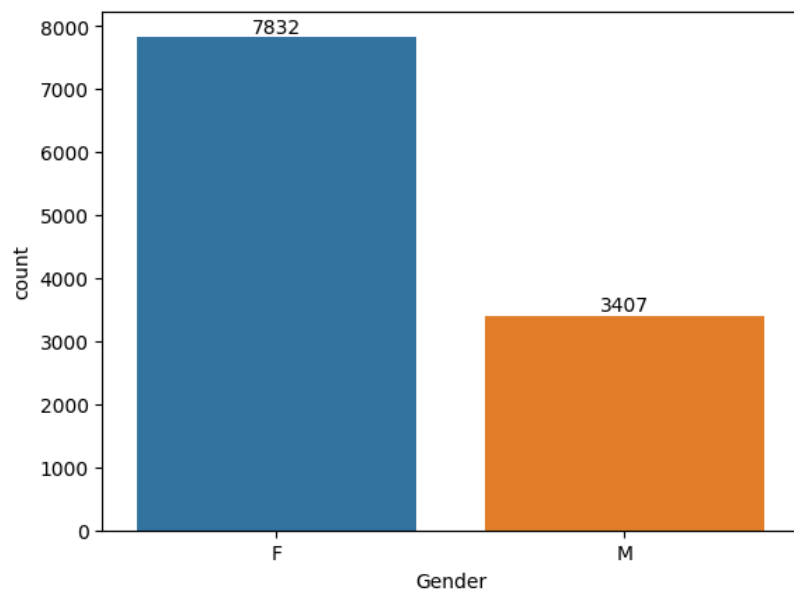
Exploratory Data Analysis

Gender

```
In [15]: # plotting a bar chart for Gender and it's count
```

```
ax = sns.countplot(x = 'Gender', data = df)
```

```
for bars in ax.containers:
    ax.bar_label(bars)
```

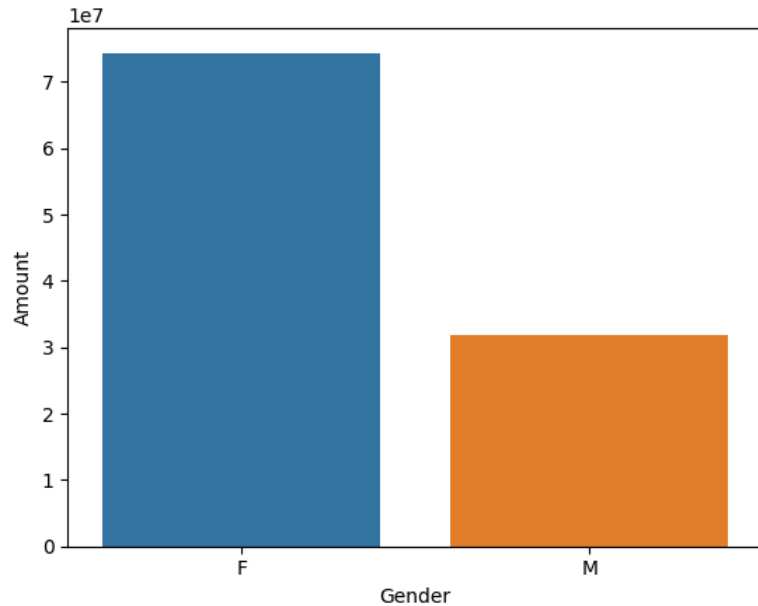


```
In [16]: # plotting a bar chart for gender vs total amount

sales_gen = df.groupby(['Gender'], as_index=False)['Amount'].sum().sort_values(by='Amount', ascending=False)

sns.barplot(x = 'Gender',y= 'Amount' ,data = sales_gen)
```

```
Out[16]: <Axes: xlabel='Gender', ylabel='Amount'>
```

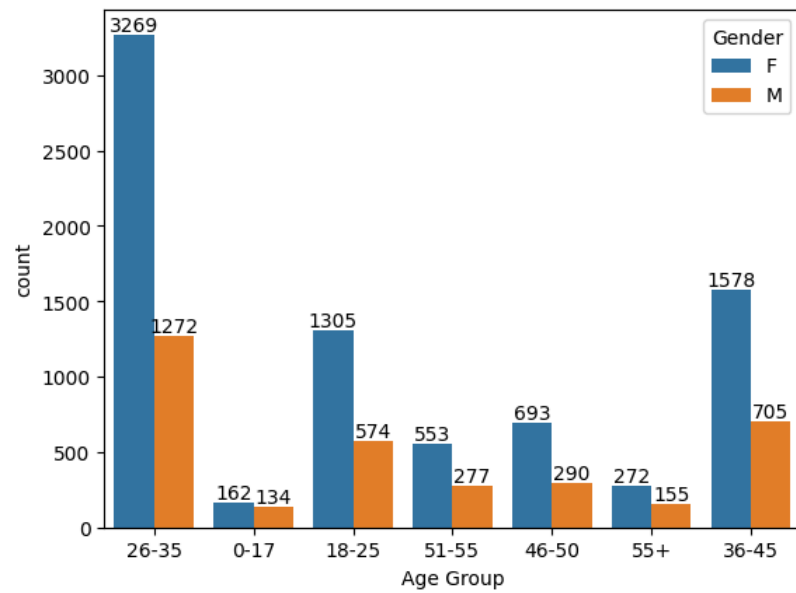


<p>From above graphs we can see that most of the buyers are females and even the purchasing power of females are greater than men</p>

<h3 id="Age">Age</h3>

```
In [17]: ax = sns.countplot(data = df, x = 'Age Group', hue = 'Gender')

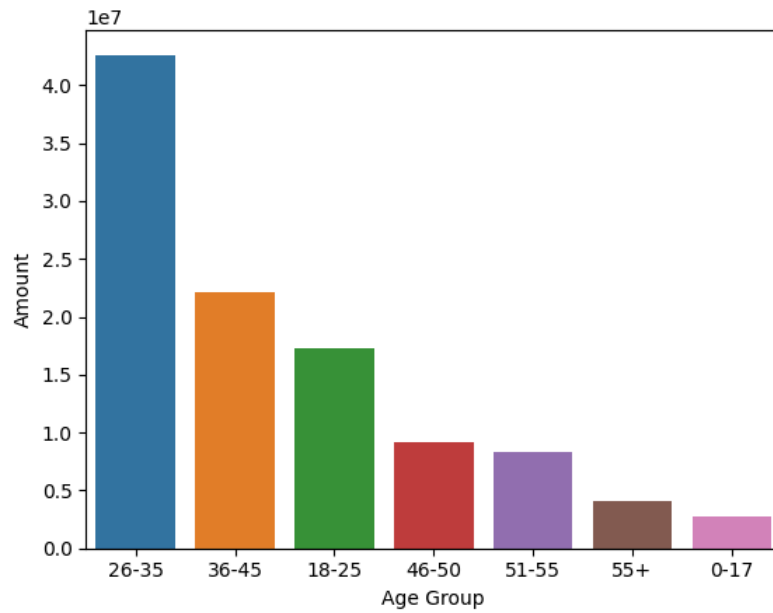
for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [18]: # Total Amount vs Age Group
sales_age = df.groupby(['Age Group'], as_index=False)['Amount'].sum().sort_values(by='Amount', ascending=False)

sns.barplot(x = 'Age Group',y= 'Amount' ,data = sales_age)
```

```
Out[18]: <Axes: xlabel='Age Group', ylabel='Amount'>
```



<p>From above graphs we can see that most of the buyers are of age group between 26-35 yrs female</p>

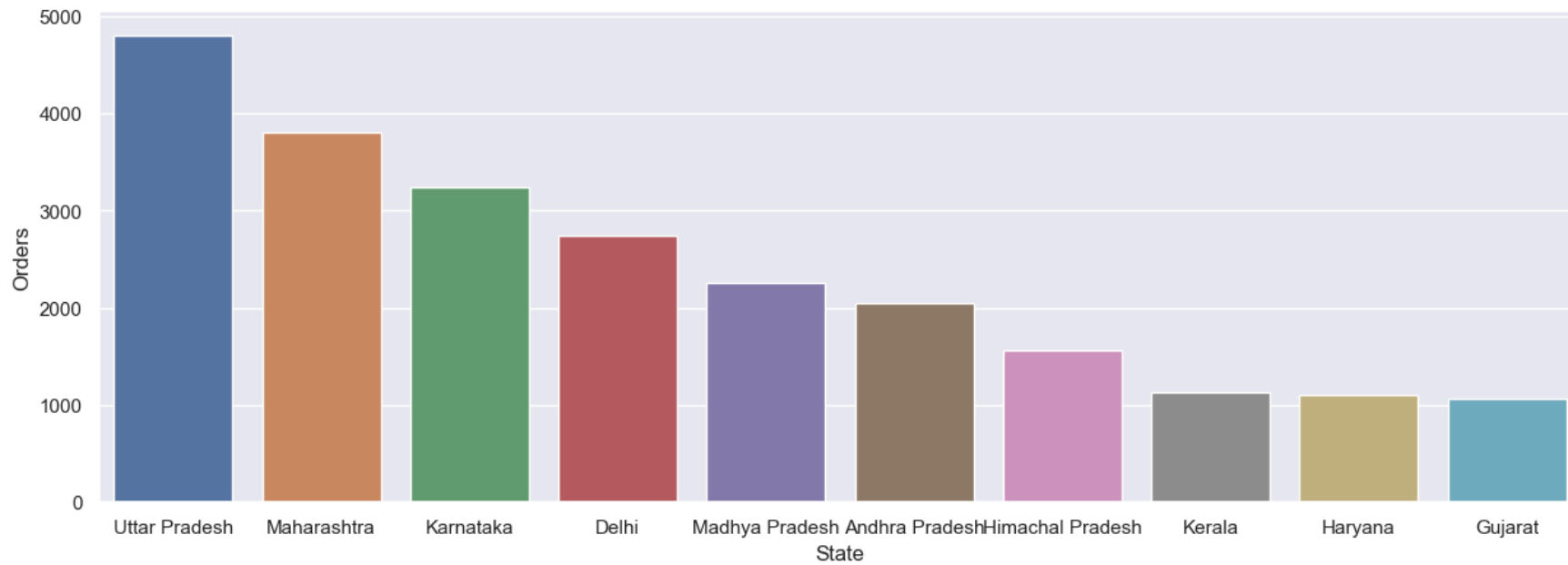
<h3 id="State">State</h3>

```
In [19]: # total number of orders from top 10 states

sales_state = df.groupby(['State'], as_index=False)['Orders'].sum().sort_values(by='Orders', ascending=False).head(10)

sns.set(rc={'figure.figsize':(15,5)})
sns.barplot(data = sales_state, x = 'State',y= 'Orders')
```

Out[19]: <Axes: xlabel='State', ylabel='Orders'>



```
In [20]: # total amount/sales from top 10 states

sales_state = df.groupby(['State'], as_index=False)['Amount'].sum().sort_values(by='Amount', ascending=False).head(10)

sns.set(rc={'figure.figsize':(15,5)})
sns.barplot(data = sales_state, x = 'State', y= 'Amount')
```

```
Out[20]: <Axes: xlabel='State', ylabel='Amount'>
```

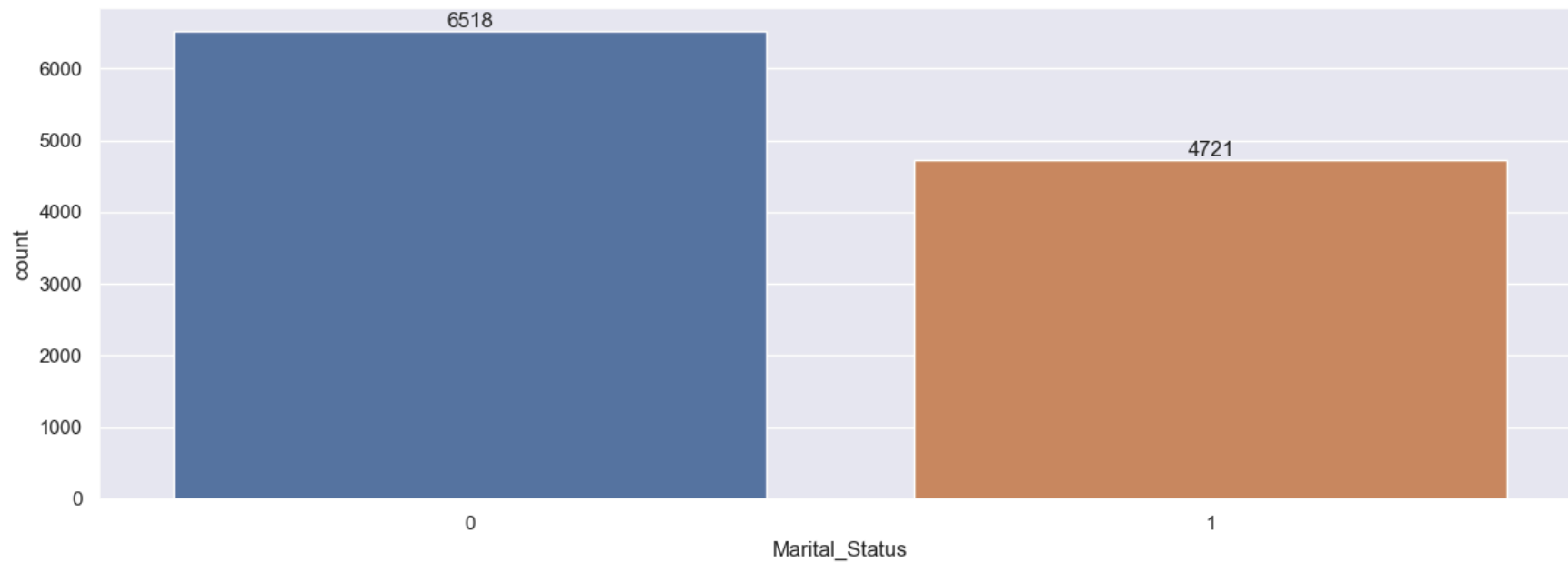


<p>From above graphs we can see that most of the orders & total sales/amount are from Uttar Pradesh, Maharashtra and Karnataka respectively</p>

<h3 id="Marital-Status">Marital Status</h3>

```
In [21]: ax = sns.countplot(data = df, x = 'Marital_Status')

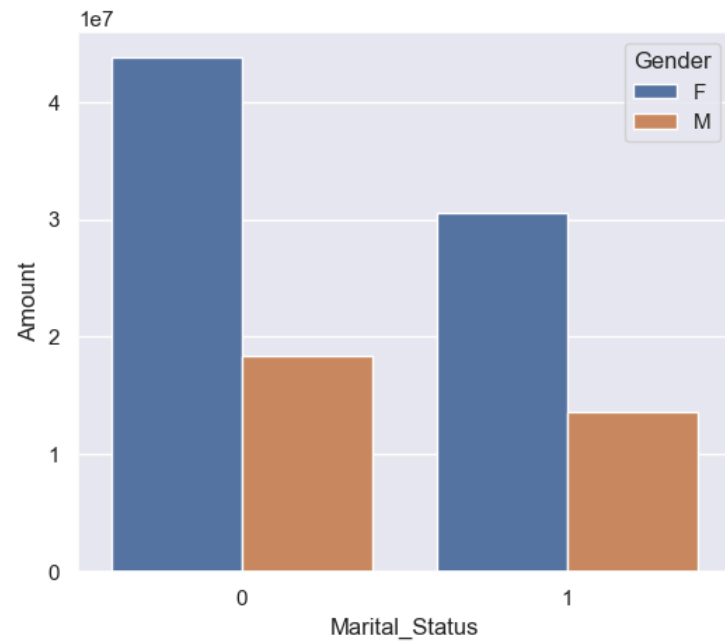
sns.set(rc={'figure.figsize':(7,5)})
for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [22]: sales_state = df.groupby(['Marital_Status', 'Gender'], as_index=False)['Amount'].sum().sort_values(by='Amount', ascending=False)

sns.set(rc={'figure.figsize':(6,5)})
sns.barplot(data = sales_state, x = 'Marital_Status',y= 'Amount', hue='Gender')
```

```
Out[22]: <Axes: xlabel='Marital_Status', ylabel='Amount'>
```

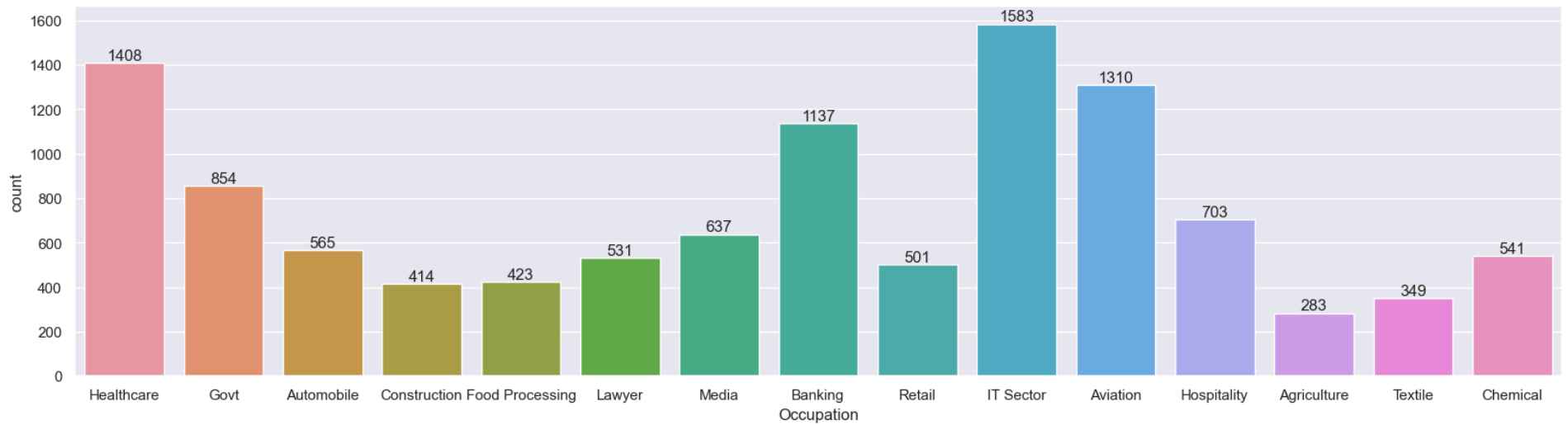


<p>From above graphs we can see that most of the buyers are married (women) and they have high purchasing power</p>

<h3 id="Occupation">Occupation</h3>

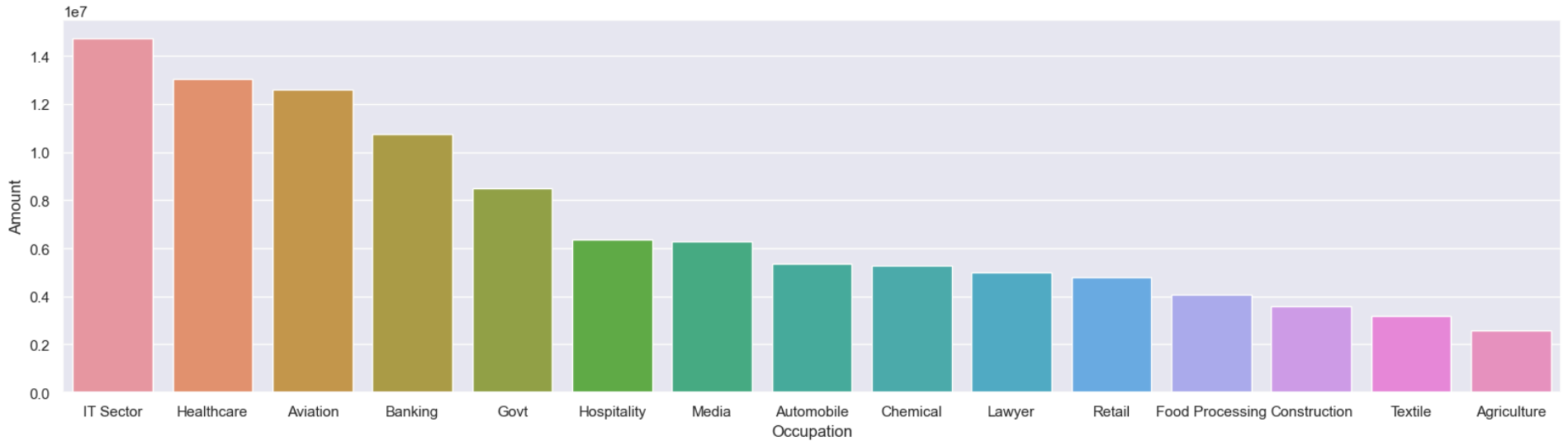
```
In [23]: sns.set(rc={'figure.figsize':(20,5)})
ax = sns.countplot(data = df, x = 'Occupation')

for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [24]: sales_state = df.groupby(['Occupation'], as_index=False)['Amount'].sum().sort_values(by='Amount', ascending=False)
sns.set(rc={'figure.figsize':(20,5)})
sns.barplot(data = sales_state, x = 'Occupation',y= 'Amount')
```

```
Out[24]: <Axes: xlabel='Occupation', ylabel='Amount'>
```

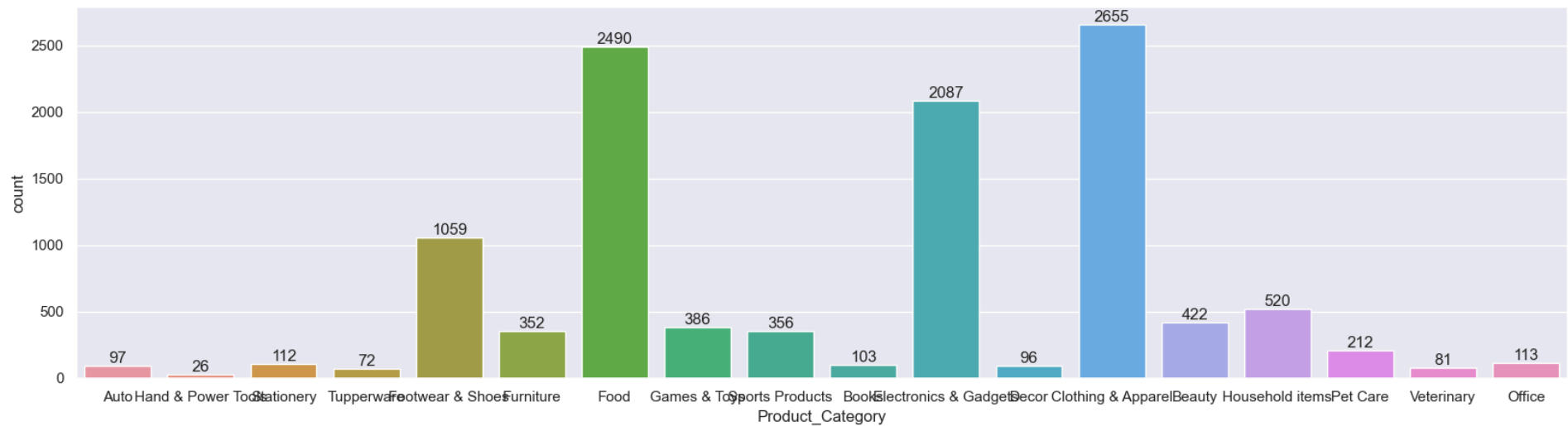


<p>From above graphs we can see that most of the buyers are working in IT, Healthcare and Aviation sector</p>

<h3 id="Product-Category">Product Category</h3>

```
In [25]: sns.set(rc={'figure.figsize':(20,5)})
ax = sns.countplot(data = df, x = 'Product_Category')

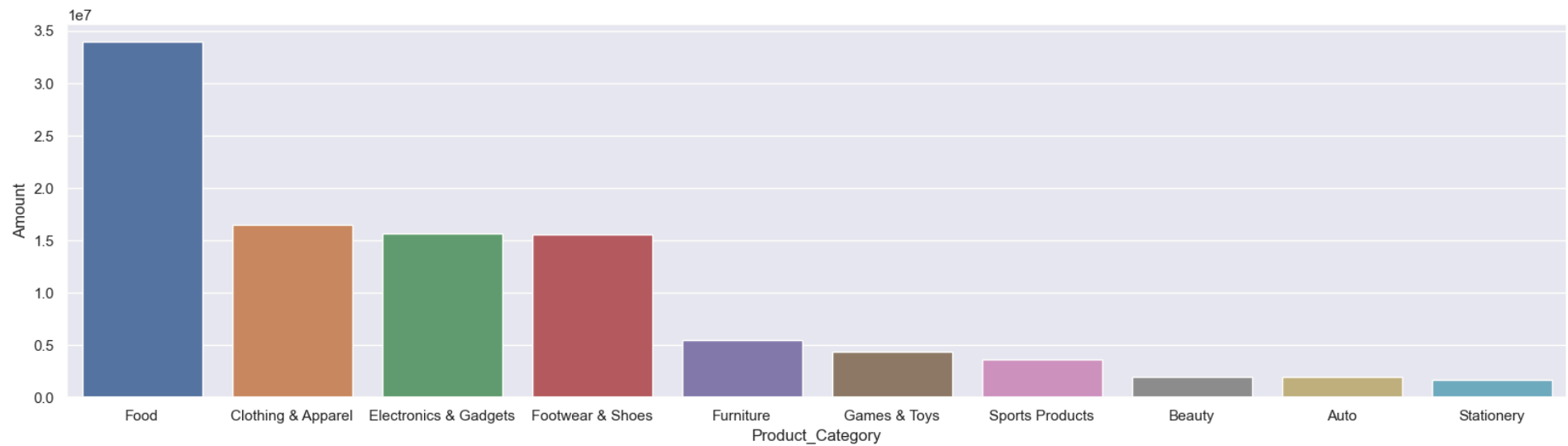
for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [26]: sales_state = df.groupby(['Product_Category'], as_index=False)['Amount'].sum().sort_values(by='Amount', ascending=False).head(10)

sns.set(rc={'figure.figsize':(20,5)})
sns.barplot(data = sales_state, x = 'Product_Category', y= 'Amount')
```

Out[26]: <Axes: xlabel='Product_Category', ylabel='Amount'>

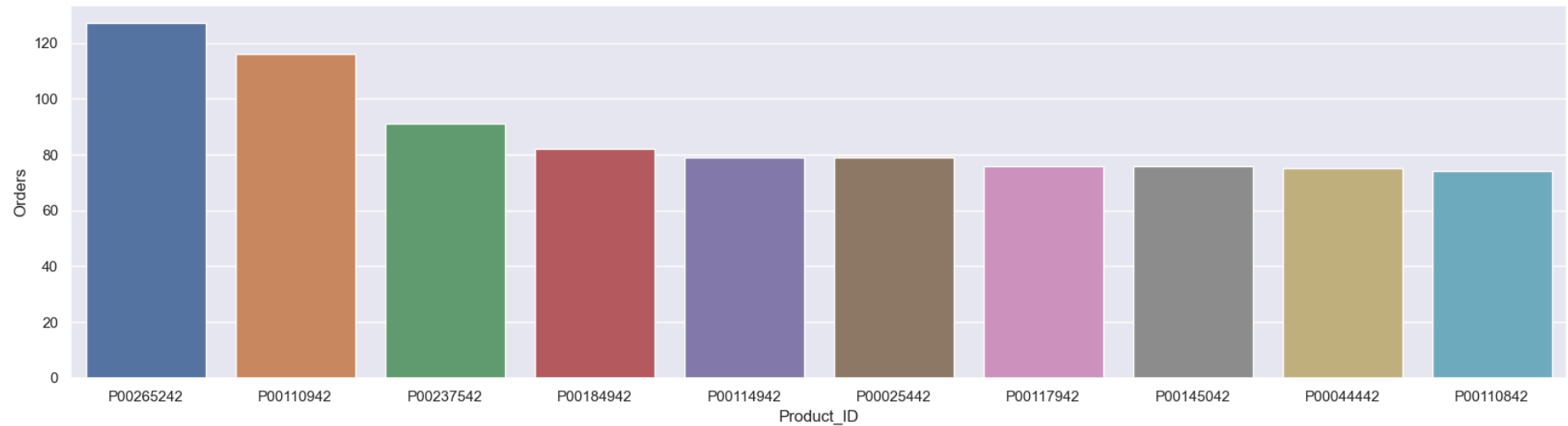


<p>From above graphs we can see that most of the sold products are from Food, Clothing and Electronics category</p>

```
In [27]: sales_state = df.groupby(['Product_ID'], as_index=False)['Orders'].sum().sort_values(by='Orders', ascending=False).head(10)

sns.set(rc={'figure.figsize':(20,5)})
sns.barplot(data = sales_state, x = 'Product_ID',y= 'Orders')
```

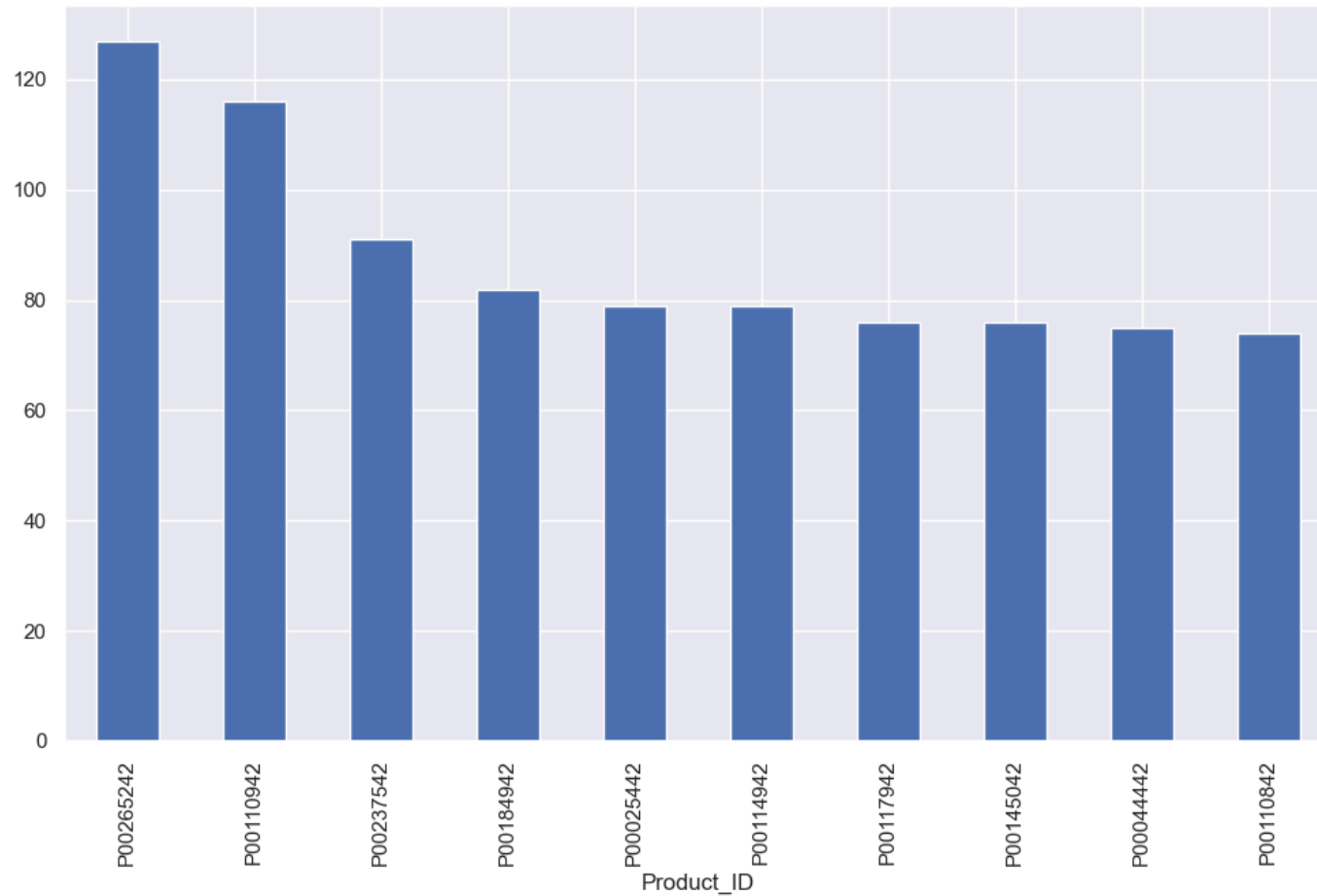
Out[27]: <Axes: xlabel='Product_ID', ylabel='Orders'>



```
In [28]: # top 10 most sold products (same thing as above)
```

```
fig1, ax1 = plt.subplots(figsize=(12,7))
df.groupby('Product_ID')['Orders'].sum().nlargest(10).sort_values(ascending=False).plot(kind='bar')
```

Out[28]: <Axes: xlabel='Product_ID'>



<h2 id="Conclusion:">Conclusion:</h2> <h3 id=""></h3>

<p>Married women age group 26-35 yrs from UP, Maharastra and Karnataka working in IT, Healthcare and Aviation are more likely to buy products from Food, Clothing and Electronics category</p>

<p>Feel free to connect with me - <https://www.linkedin.com/in/palak-tripathi-37a56234a/></p>

<p>Thank you!</p>